PBJ-2 PRACTICAL ASSIGNMENT

Camunda 8: End-to-End Workflow Automation

General Instructions

- Complete the three assignments as separate workflow implementations using Camunda 8 Modeler for BPMN and DMN models.
- Use Java with Spring Boot for external job workers and configure REST Connectors or a mock REST API for service integration.
- Deploy the required BPMN and DMN models and execute multiple workflow instances with different test inputs.
- Use Tasklist for human tasks and Operate to inspect process instances, variables, failures, and incidents.
- Use synthetic test data. Do not include real payment credentials, secrets, or real patient data in the repository.
- Submit model files, source code, API examples, test evidence, screenshots, and a concise implementation report for each lab.
- Add screenshots captured from your own running implementation. Do not submit placeholder images as evidence.

LAB 1: End-to-End Loan Approval Automation

Problem Statement: Develop an end-to-end loan approval workflow using Camunda 8. The solution must combine BPMN orchestration, DMN-based decisions, an external job worker, REST integration, process variables, human review, monitoring, and incident recovery.

Requirements

1. Create a BPMN process that accepts applicant details, requested loan amount, monthly income, and relevant application information.
2. Create a DMN decision table that classifies an application as Approved, Rejected, or Manual Review using documented educational rules.
3. Implement a Java/Spring Boot external job worker to validate applicant information and return the validation result as process variables.
4. Configure a REST Connector to call a simulated or actual credit-assessment API. Use synthetic data for testing.
5. Pass applicant details, validation results, credit-assessment results, and decision outcomes between activities using process variables.
6. Use gateways to route the application according to the DMN result and service responses.
7. Assign a manual review user task through Tasklist for applications that require additional review.
8. Deploy the BPMN and DMN models and test multiple application instances with different input values.
9. Use Operate to inspect process execution, variables, and activity states. Intentionally cause one service/worker failure, diagnose the incident, correct its cause, and retry the failed job where supported.

Minimum Test Scenarios

- Application meeting the defined approval criteria.
- Application failing the defined eligibility criteria.
- Application requiring manual review.
- Credit-assessment API error or timeout.
- Worker failure followed by correction and successful recovery.

Expected Deliverables

- BPMN diagram and deployed process definition.
- DMN decision table with input/output definitions and sample evaluations.
- Spring Boot worker source code and configuration instructions.
- REST request/response example and process-variable list.
- Test-case table showing inputs, expected route, actual outcome, and status.
- Tasklist and Operate screenshots, including incident investigation and recovery.
- Brief report describing architecture, implementation, results, and limitations.

LAB 2: End-to-End E-Commerce Order Processing and Fulfilment

Problem Statement: Build an order-processing workflow that validates an order, checks stock, determines priority, processes a simulated payment, and routes the order to fulfilment or review.

Requirements

1. Design a BPMN process that receives customer reference, product ID, quantity, delivery preference, and payment details or a payment token for a mock service.
2. Create a DMN decision table that determines order priority using order value, customer category, and delivery preference. Document the rules and outputs.
3. Implement an external job worker using Java and Spring Boot to validate the order, check simulated inventory, and calculate the order total.
4. Configure a REST Connector to invoke a payment gateway API or mock payment service. Do not use real payment credentials.
5. Use process variables to exchange order details, stock status, order total, payment status, and delivery priority.
6. Use exclusive gateways to route orders based on stock availability and payment outcome.
7. Create a Tasklist user task for manual verification of orders meeting a defined high-value or exception condition.
8. Deploy the BPMN and DMN models and run multiple workflow instances covering normal and exceptional paths.
9. Monitor instances in Operate. Simulate a payment or inventory-service failure, inspect the incident, fix the underlying cause, and retry the failed job where supported.

Minimum Test Scenarios

- Stock available and payment succeeds.
- Stock insufficient and order cannot proceed to fulfilment.
- Payment declined by the mock service.
- High-value order routed to manual review.
- Payment or inventory service fails and workflow recovers after corrective action.

Expected Deliverables

- BPMN diagram and DMN priority decision table.
- Spring Boot worker source code and setup instructions.
- REST Connector configuration and sample API request/response.
- Process-variable dictionary and gateway-routing explanation.
- Test-case table with expected and actual results.
- Tasklist screenshots showing review tasks and Operate screenshots showing execution and incident recovery.
- Brief report documenting implementation and assumptions.

LAB 3: End-to-End Hospital Appointment and Patient Triage Management

Problem Statement: Develop a hospital appointment workflow that registers a patient, applies predefined educational triage rules, checks appointment availability, routes cases for staff review, and supports monitoring and recovery. Use synthetic patient data and a mock scheduling API.

Requirements

1. Design a BPMN process to collect a synthetic patient reference, appointment preference, symptoms summary, and predefined urgency indicators.
2. Create a DMN decision table that classifies requests as Routine, Priority Review, or Urgent Assessment using predefined rules for the educational scenario.
3. Implement a Java/Spring Boot external job worker to validate registration fields and request appointment availability.
4. Configure a REST Connector to call a mock hospital scheduling API and retrieve available appointment slots.
5. Use process variables to exchange patient reference, appointment preference, urgency category, available slots, and booking status.
6. Use gateways to route requests based on the simulated triage category and appointment availability.
7. Create a Tasklist user task for authorised staff review when a request requires manual intervention or no suitable slot is available.
8. Deploy and execute multiple workflow instances, then inspect process status, variables, and activity history in Operate.
9. Simulate a scheduling-service failure, inspect the resulting incident, correct the cause, and retry the failed job where supported.

Minimum Test Scenarios

- Routine request with an available appointment slot.
- Request routed for priority staff review.
- Urgent-assessment category routed to immediate human assessment in the simulated workflow.
- No appointment slots available.
- Scheduling API fails and workflow is recovered after corrective action.

Expected Deliverables

- BPMN diagram and DMN decision table with documented educational rules.
- Spring Boot worker source code and mock scheduling API details.
- REST Connector configuration and sample request/response.
- Process-variable dictionary and routing explanation.
- Test-case table with expected and actual results.
- Tasklist and Operate screenshots showing workflow execution and incident recovery.
- Brief report describing implementation, results, assumptions, and limitations.

Common Submission Checklist

☐ BPMN model files and screenshots of process design.

☐ DMN model files, rule descriptions, and sample decision results.

☐ External job worker source code and instructions to run it.

☐ REST Connector configuration and API examples.

☐ Process-variable dictionary including variable names, types, and purpose.

☐ Test cases covering successful, rejected, manual-review, and failure/recovery paths.

☐ Evidence of model deployment and workflow instance execution.

☐ Tasklist evidence for human tasks.

☐ Operate evidence for monitoring and incident resolution.

☐ Short report explaining architecture, execution flow, results, and limitations.